ブランチ操作

慶應義塾大学理工学部物理情報工学科
渡辺

今回の目標

- Gitでのブランチ操作、特にブランチの必要性とワークフローについて理解する
- マージと衝突の解決について理解する
- リベースについて理解する

特に「歴史を分岐、改変する」とはどういうことか、なぜ行うかを理解する

なぜブランチを分けるか

AliceとBobの二人が開発に関わるプロジェクト

Alice

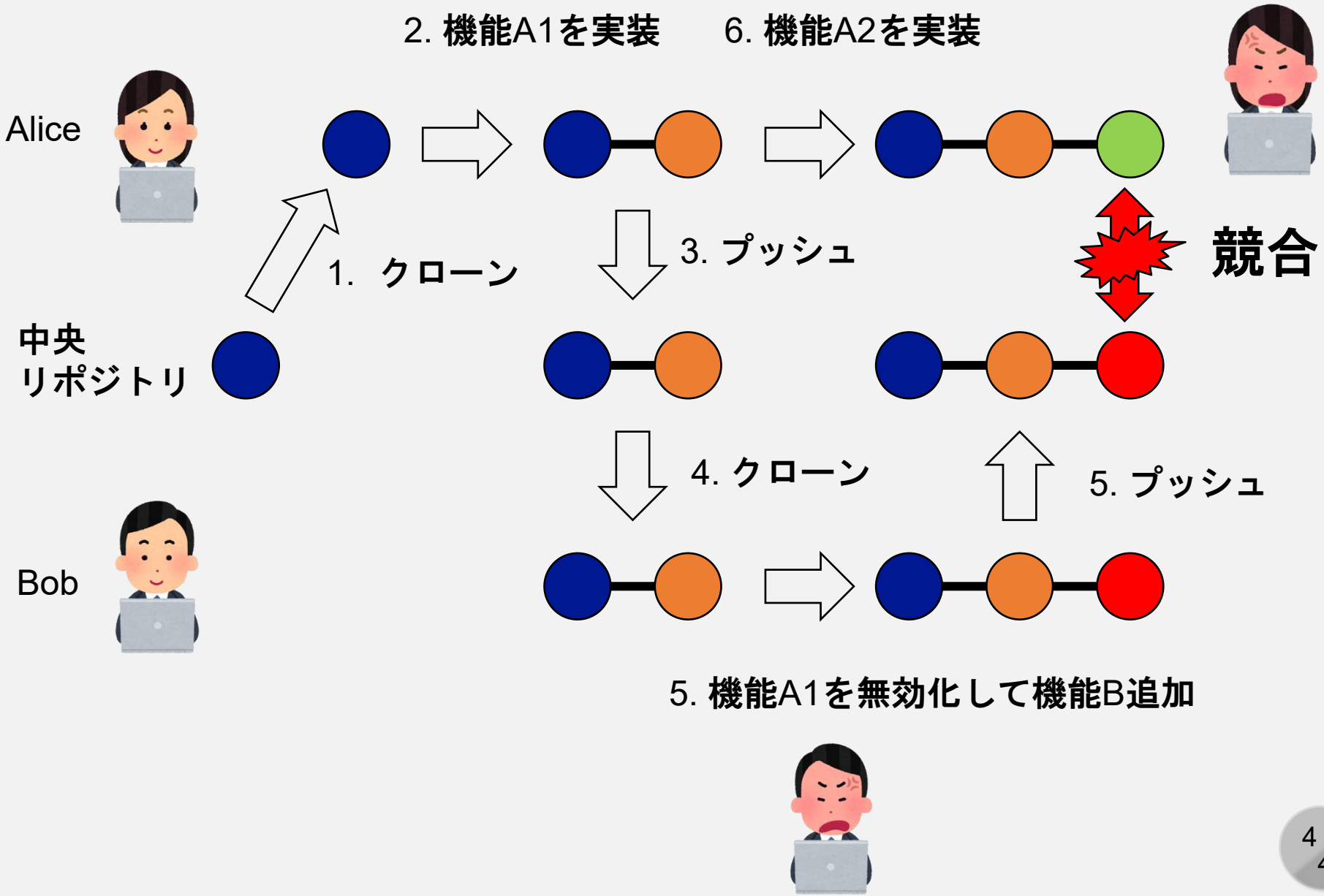
機能Aを開発

- 機能Aの達成のためには、サブモジュールA1とA2が必要
- A1を実装しただけでは正しく動作しない
- A2まで実装して初めて全体として正しく動作する

Bob

機能Bを開発

なぜブランチを分けるか



なぜブランチを分けるか

何が問題だったか？

Aliceが機能A1を実装した状態という「中途半端な状態」をコミットした
Bobはその状態をクローン、動作しないのでA1を無効化せざるを得なかった

どうすれば良い？

案1: 「中途半端な状態をコミットしてはならぬ」というルールを作る
案2: ブランチを作成して自由にコミットするが、マージの時にルールを作る

➡ ワークフロー(Work Flow)の導入

以下の動画で、Linusが「コミットルール」について触れている

Tech Talk: Linus Torvals on git

https://www.youtube.com/watch?v=4XpnKHJAok8

ワークフローとは

多人数開発時のためのGitのブランチ運用ルール

みんなが参照するブランチに中途半端な状態は困る
個人のブランチは好き勝手やってよい
メインブランチと開発ブランチをどう整理するか？

有名なワークフロー

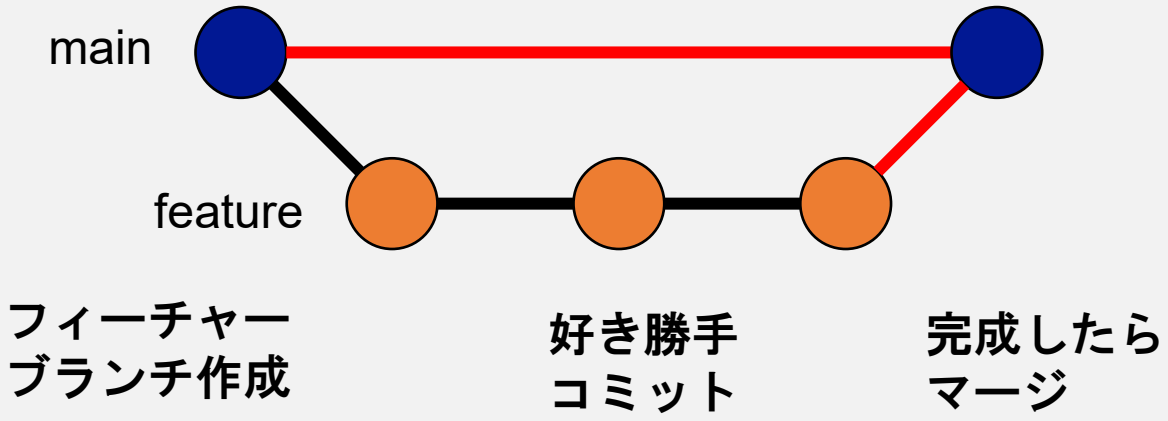
- フィーチャーブランチワークフロー
- Git Flow
- GitHub Flow
- GitLab Flow

↑本講義ではこれを説明

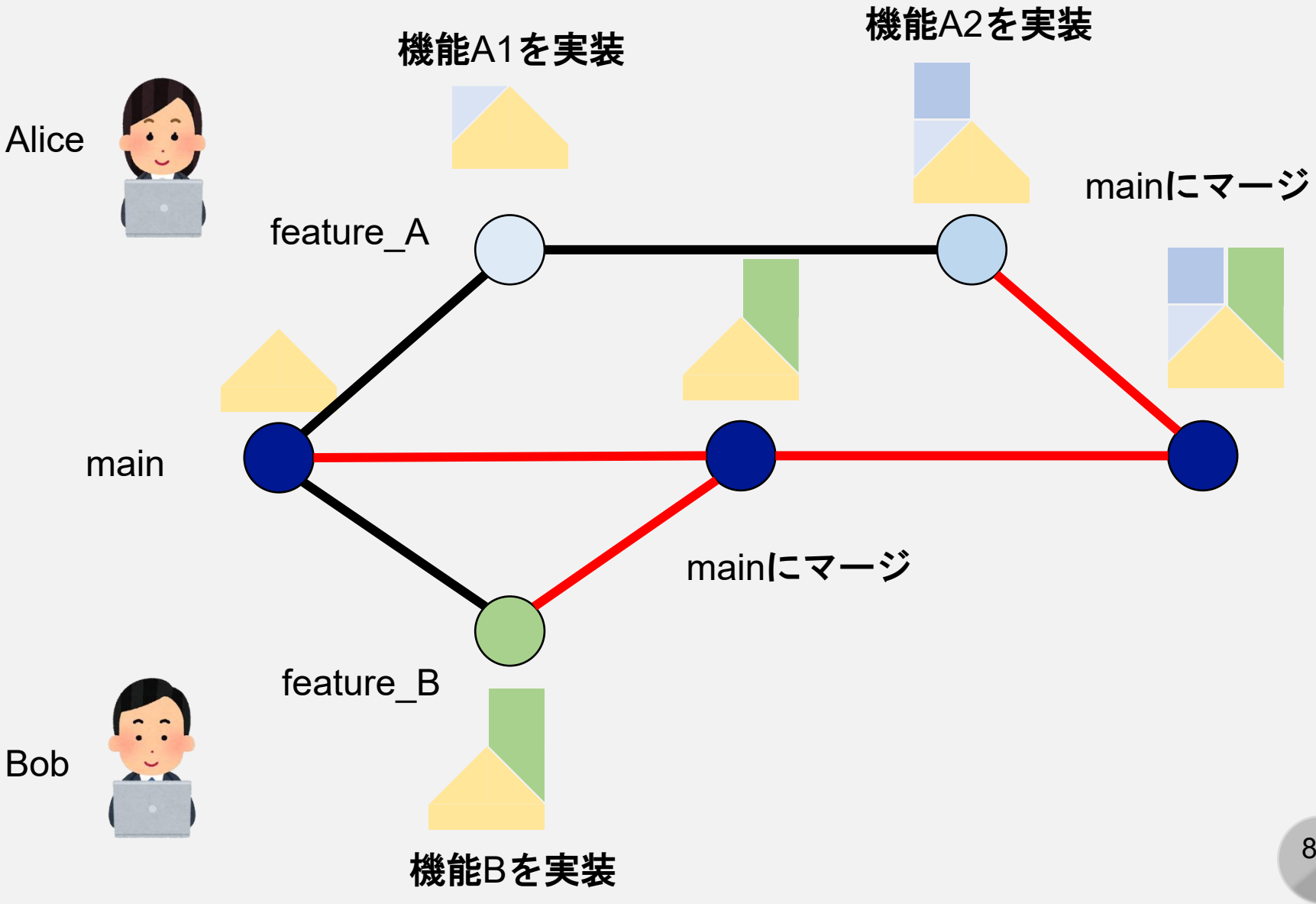
フィーチャーブランチWF

フィーチャーブランチワークフローとは

- シンプルなワークフロー
- mainブランチで作業しない
- 機能ごとにブランチを作成(フィーチャーブランチ)
- 機能が完成したらマージ



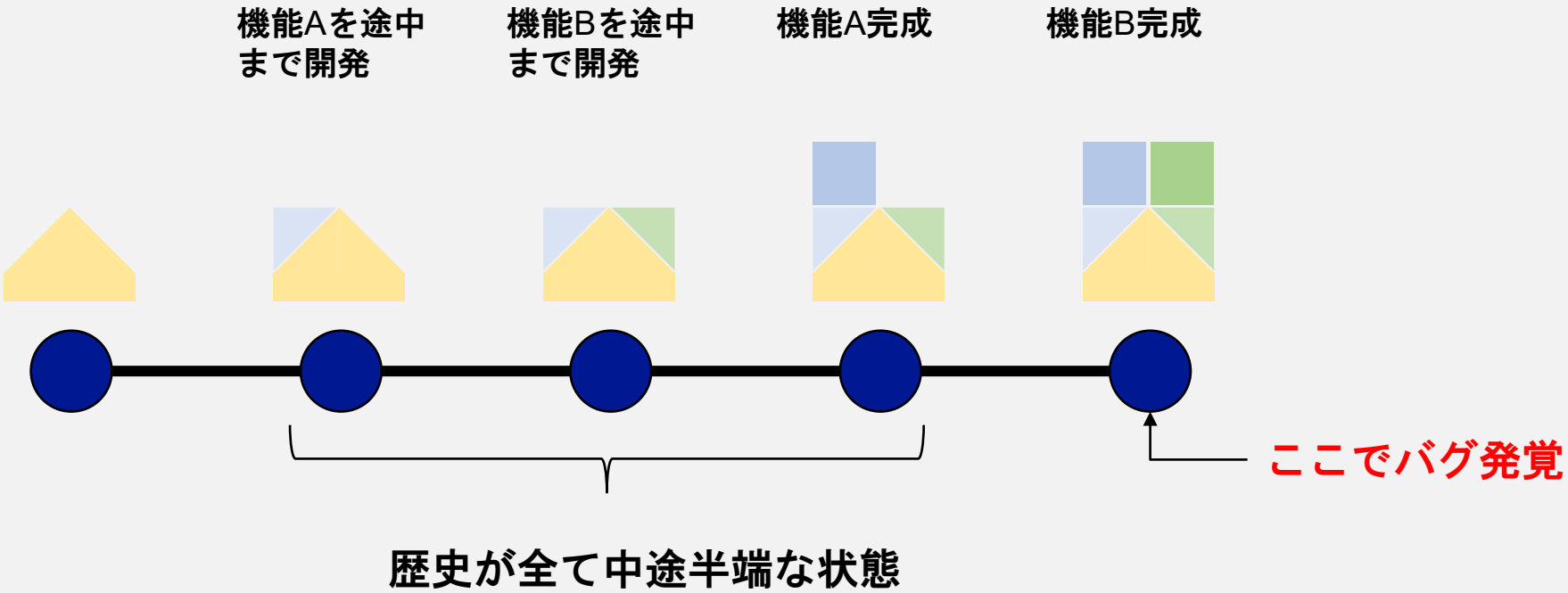
フィーチャーブランチWF



個人開発とワークフロー

ワークフローは複数人開発のためのルールだが、個人開発でも有用

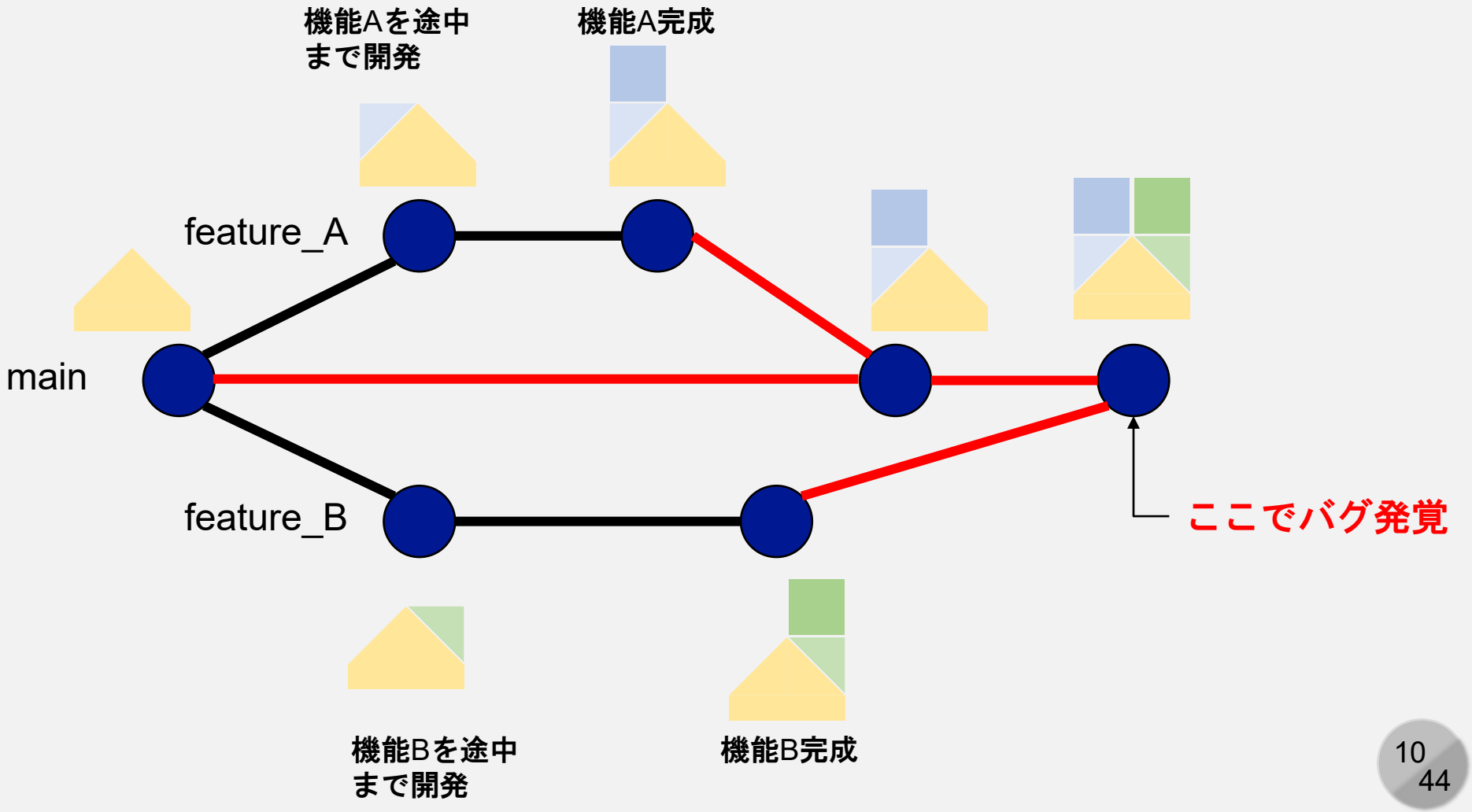
もしあなたが「まっすぐ一本な歴史」で開発していたら？



どのコミットがバグに寄与しているのかわかりづらい

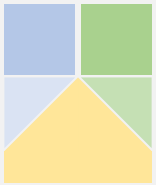
個人開発とワークフロー

もし「機能ごとにブランチを派生させる」ルールを守っていたら？

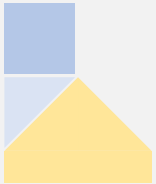


個人開発とワークフロー

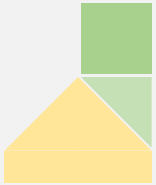
もし「機能ごとにブランチを派生させる」ルールを守っていたら？



機能AとBが両方実装された状態



機能Aのみ実装された状態



機能Bのみ実装された状態

これらを調べることで
容疑者を限定できる

単独開発でも多人数開発のルールを導入した方がよい

三日前の自分は他人

ワークフローのまとめ

なぜブランチを分けるか

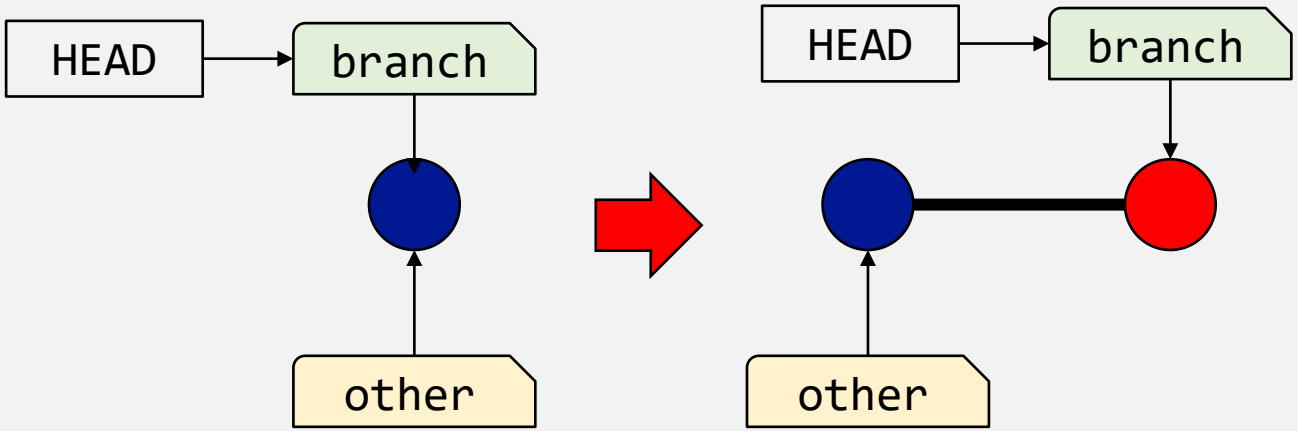
- いま行っている作業は何かを明確にするため
- 複数の作業を同時に行わない
- Gitは便利ツールではなく「作業フロー」を実現するのを助けてくれるツール
- 「作業フロー」を変えなければ意味は薄い

ワークフローとは何か

- 多人数開発時のブランチ運用のルール
- 個人開発でも有用
- 基本は「mainでコミットしない」こと
- ブランチで開発、mainにマージ

カレントブランチとコミット

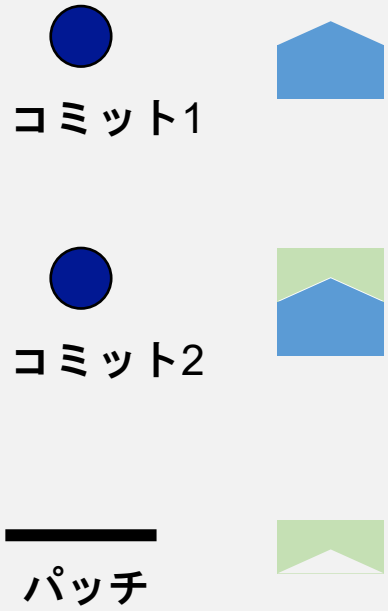
Gitの歴史：コミットがつながったもの
ブランチ：コミットについたラベル
カレントブランチ：HEADが指しているブランチ



コミットするとコミットが作られカレントブランチが指していたコミットにつながる
さらにカレントブランチが新たに作られたコミットを指す
他のブランチは動かない
ブランチを切り替えてコミットすると歴史が分岐する

コミットと差分

コミットは「自分の親コミット」を知っている
親コミットをたどることで歴史をたどることができる



コミット1からコミット2が作られた
コミットはそれぞれの時点でのスナップショット表す
コミット間の線は差分(パッチ)を表す

ブランチ作成と切り替え

ブランチの作成

```
$ git branch ブランチ名 ブランチをつけたいコミット
```

「コミット」を省略すると、カレントブランチが指すコミットに別名をつける

```
$ git branch ブランチ名
```

ブランチの切り替え

```
$ git switch ブランチ名
```

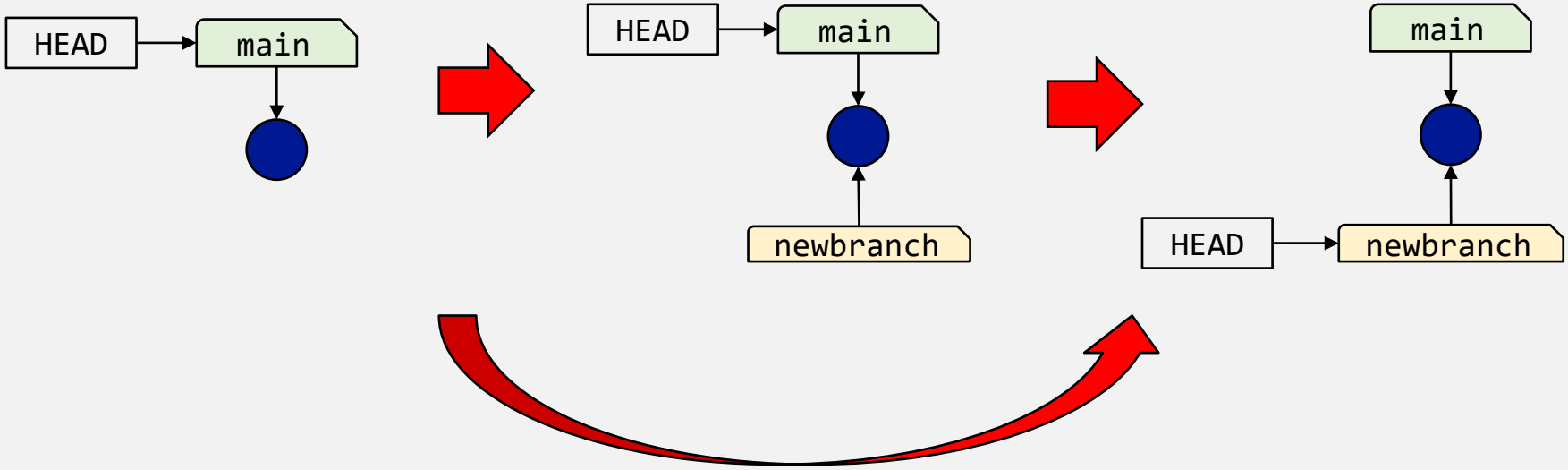
ブランチを作成して切り替える

```
$ git switch -c ブランチ名
```

ブランチ作成と切り替え

```
$ git branch newbranch
```

```
$ git switch newbranch
```




```
$ git switch -c newbranch
```

マージ

マージとは、二つのブランチの共通祖先を見つけ、そこからの修正を全て取り込んだ新たなコミットを作る作業

```
$ git merge ブランチ名
```

「カレントブランチ」に「ブランチ名」の修正を取り込む

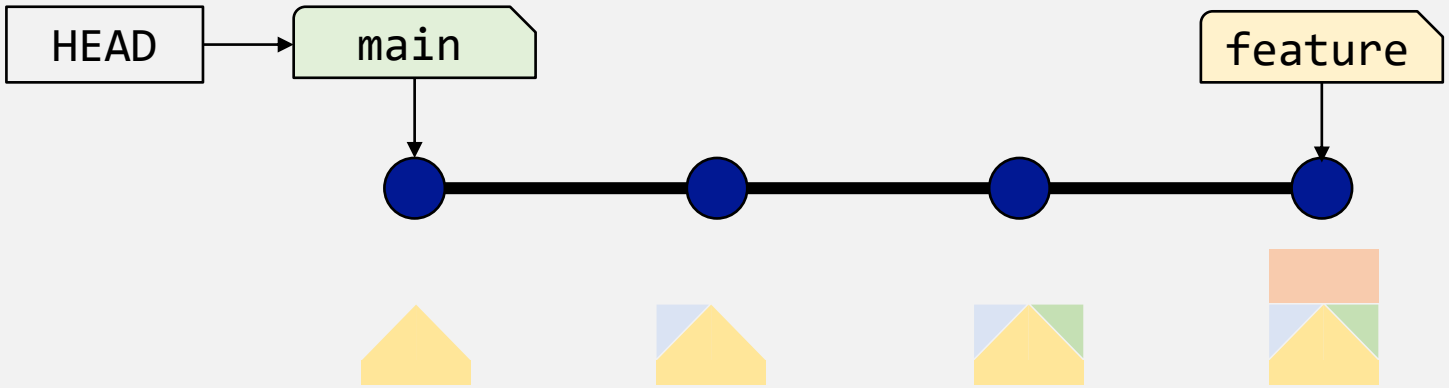
個人開発の場合、mainであることがほとんど

```
$ git merge
```

ブランチ名を省略すると「上流ブランチ」をマージ(後述)

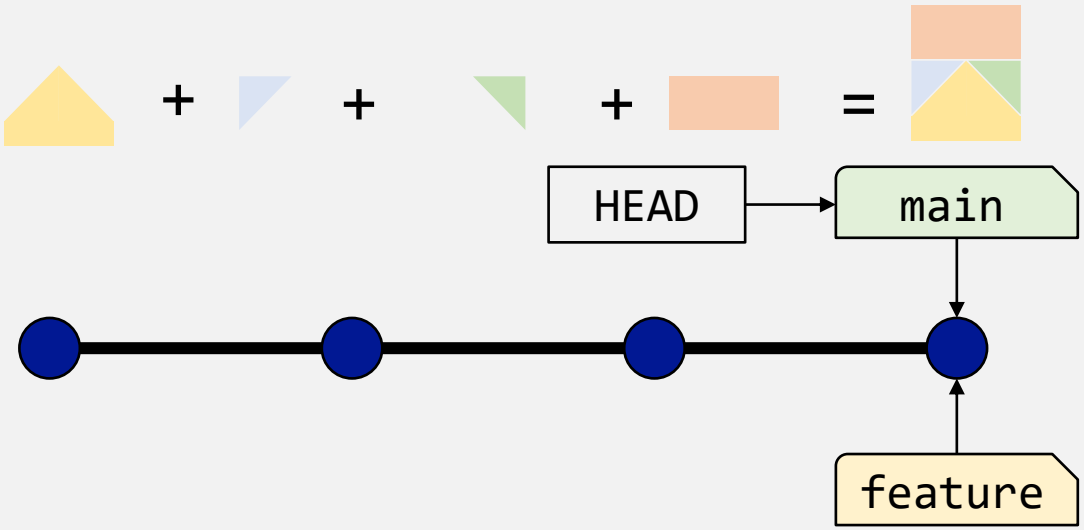
Fast-Forward マージ

カレントブランチがマージしたいブランチの直接の祖先であるとき

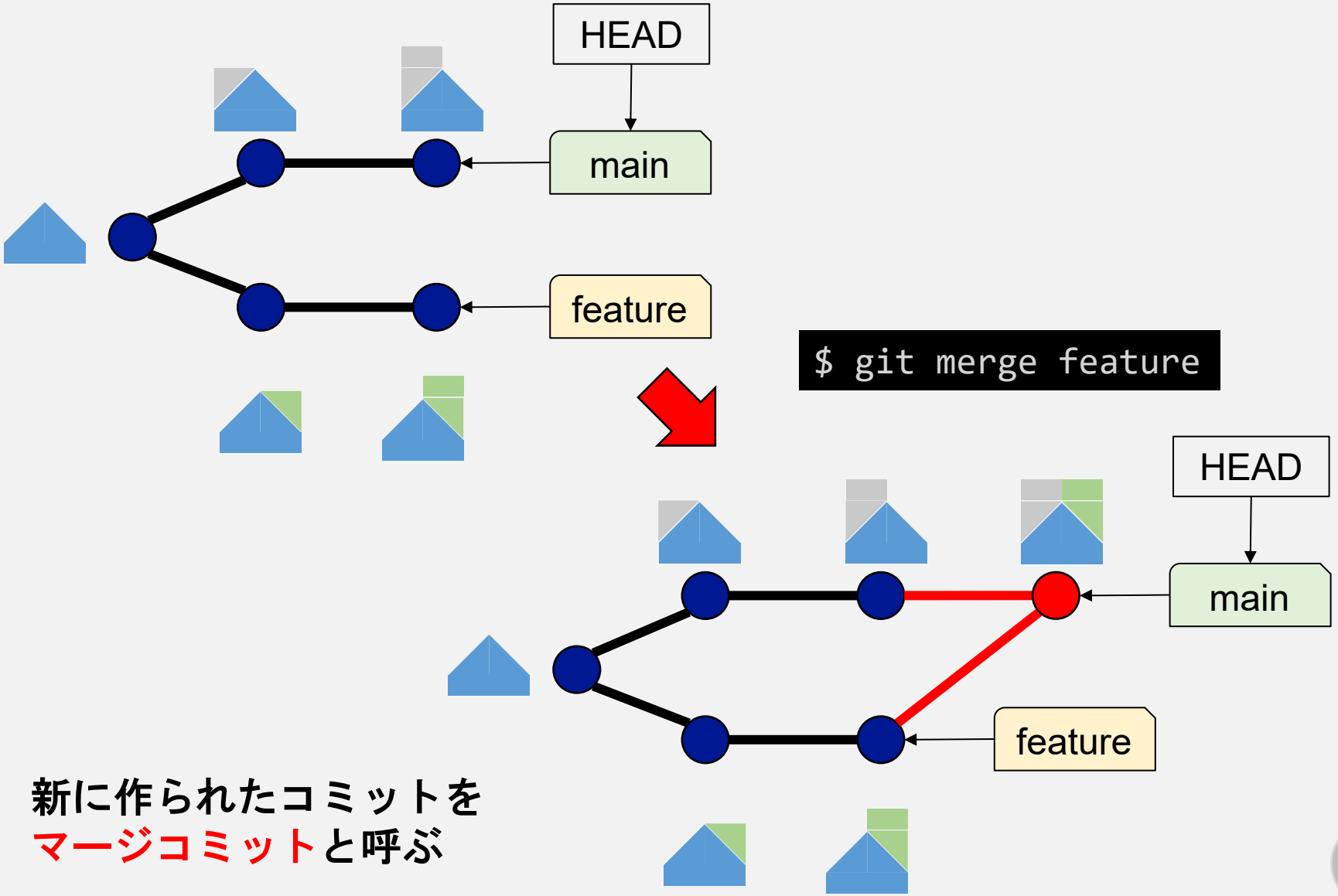



```
$ git merge feature
```

カレントブランチを
移動するだけでマージ完了



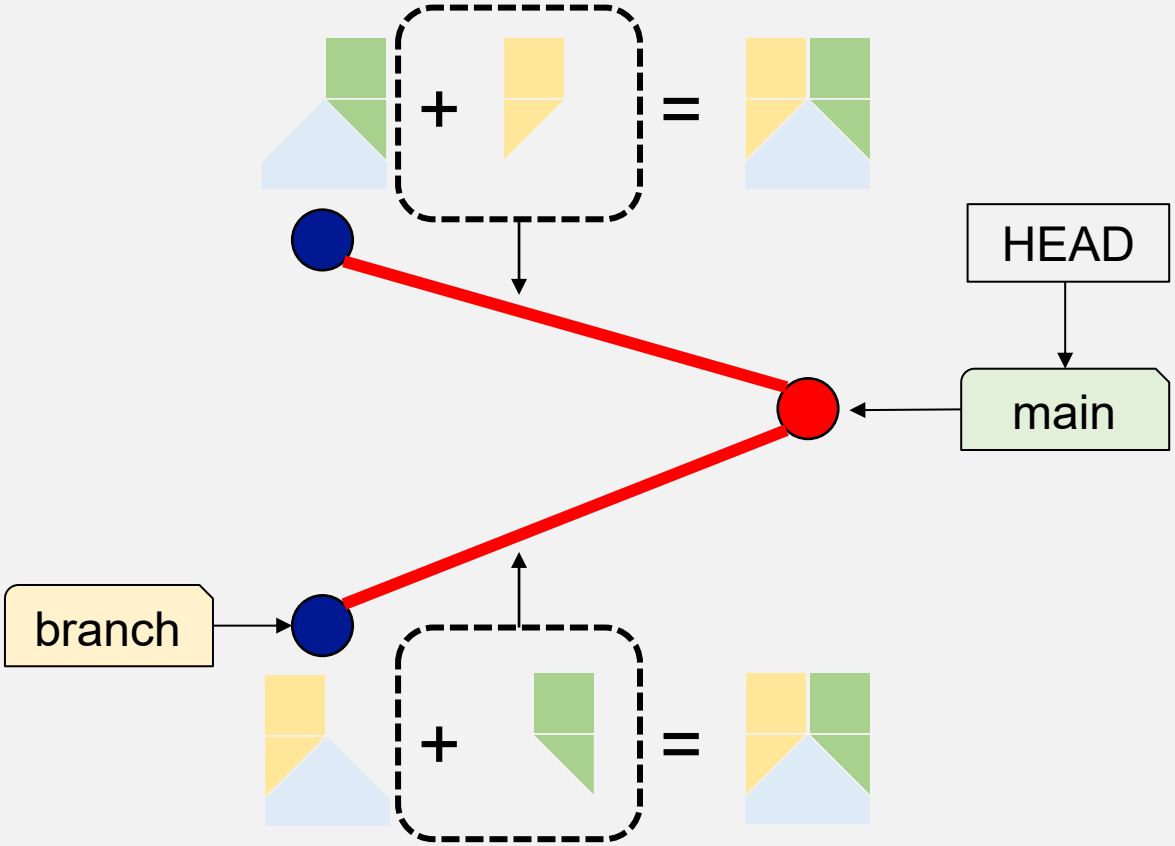
Non-fast forward マージ



新に作られたコミットを
マージコミットと呼ぶ

マージコミット

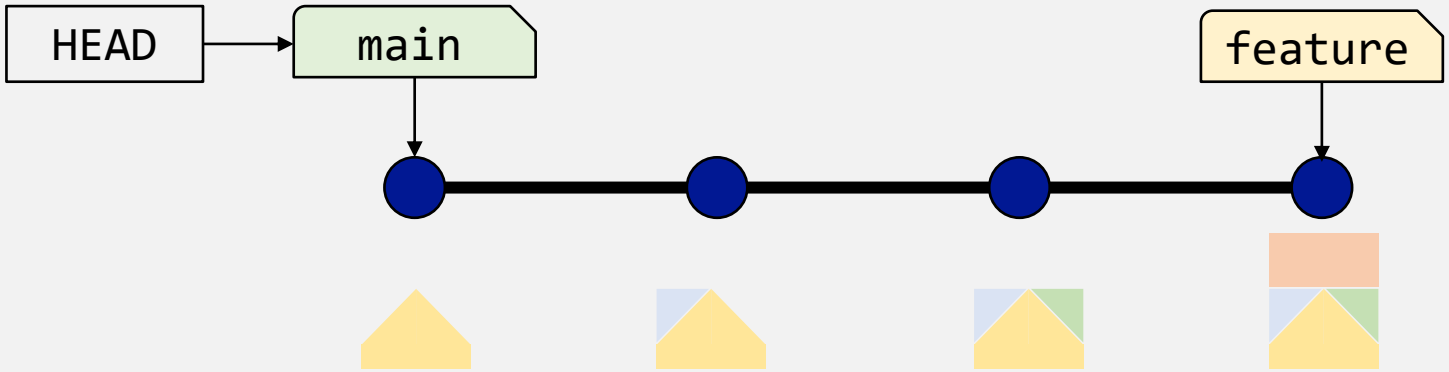
non fast-forwardマージで作られるコミット
親コミットを二つ持つ



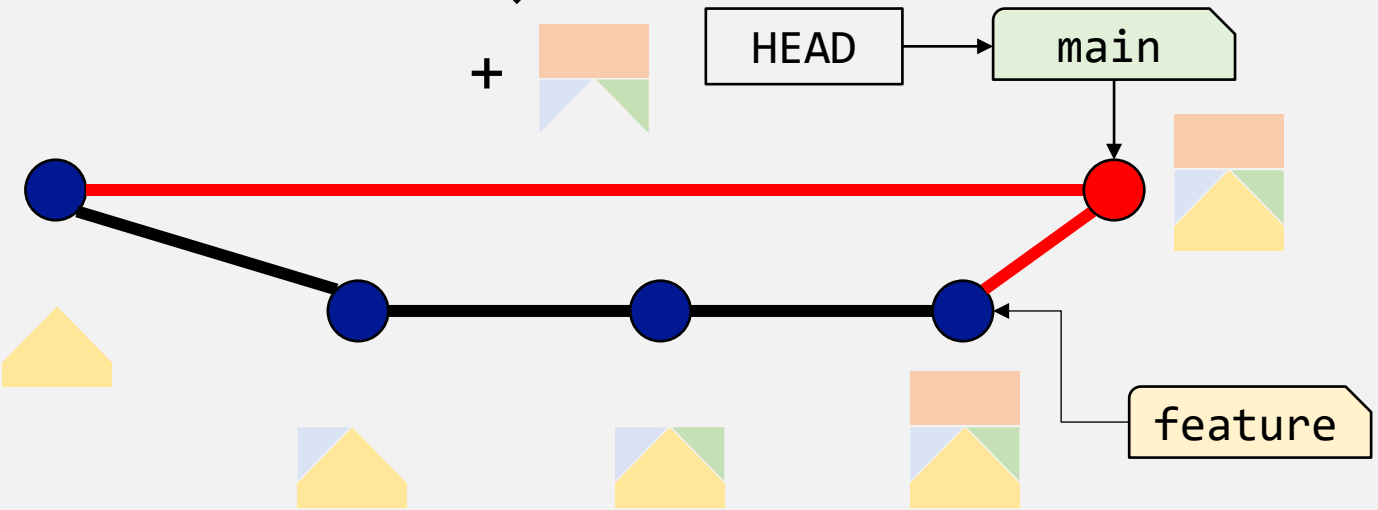
二つの線は、それぞれの親からの差分を表している

マージコミット

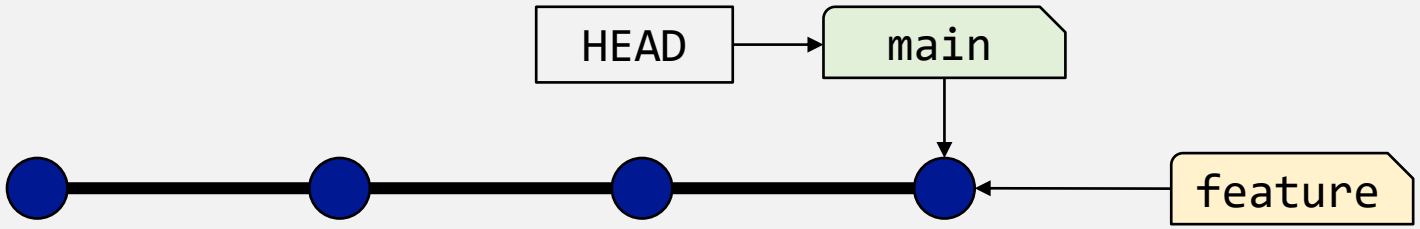
Fast-forwardマージ可能な場合でもマージコミットを作ることができる



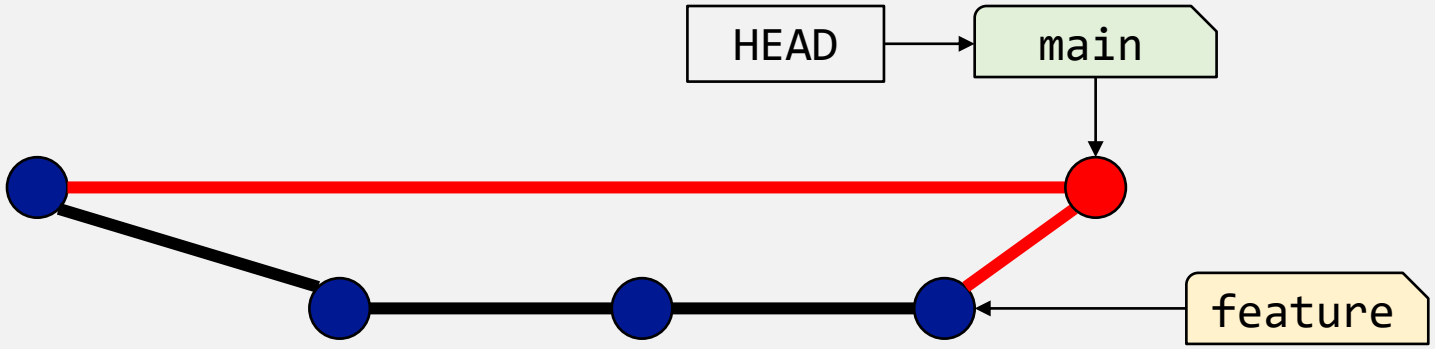

```
$ git merge --no-ff feature
```



マージコミット



fast-forwardすると歴史が一本になって見やすい
どこから分岐したかの情報が失われ、マージを取り消しづらい

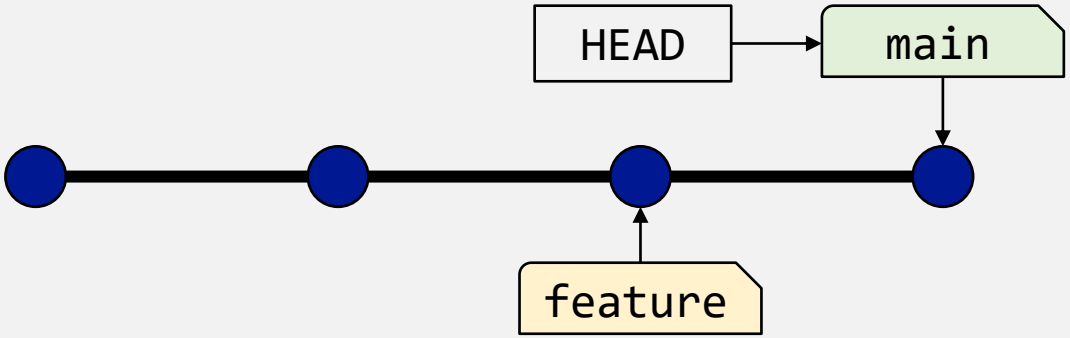


マージコミットを作ると、機能追加をしようとした
ブランチがどこから分岐したかの情報が残る

ブランチの削除

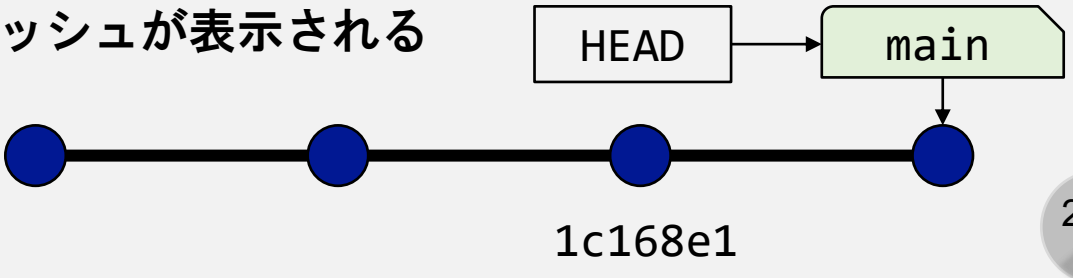
```
$ git branch -d ブランチ名
```

指定したブランチを削除する
削除されるのはブランチだけで、コミットは消えない



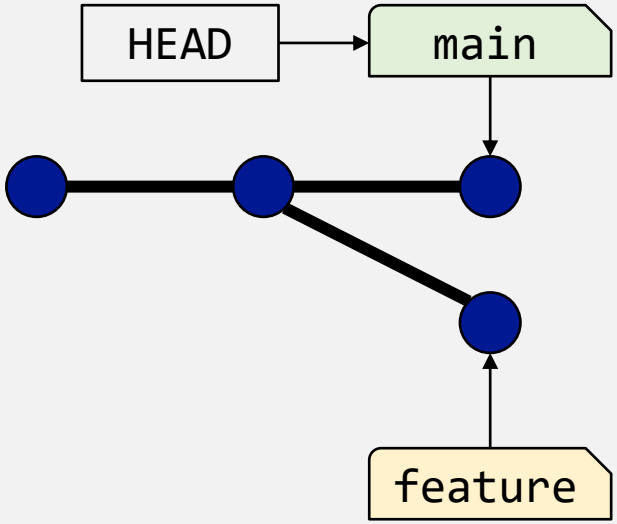

```
$ git branch -d feature
Deleted branch feature (was 1c168e1).
```

削除前に指していたコミットハッシュが表示される



ブランチの削除

マージされていないブランチを削除しようとするとエラーになる



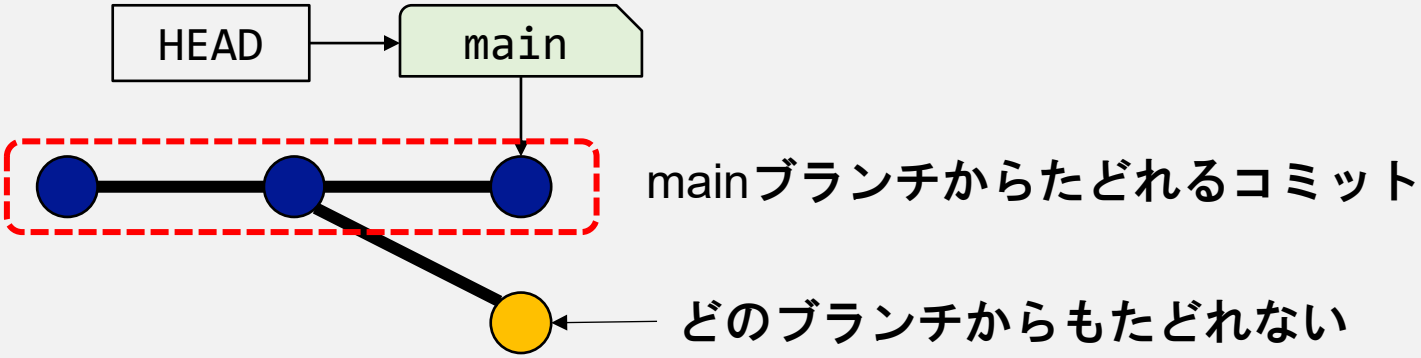

```
$ git branch -d branch
error: The branch 'feature' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feature'.
```

もしどうしても削除したいのなら「-D」オプションをつけろ

ブランチの削除

もし削除したら？

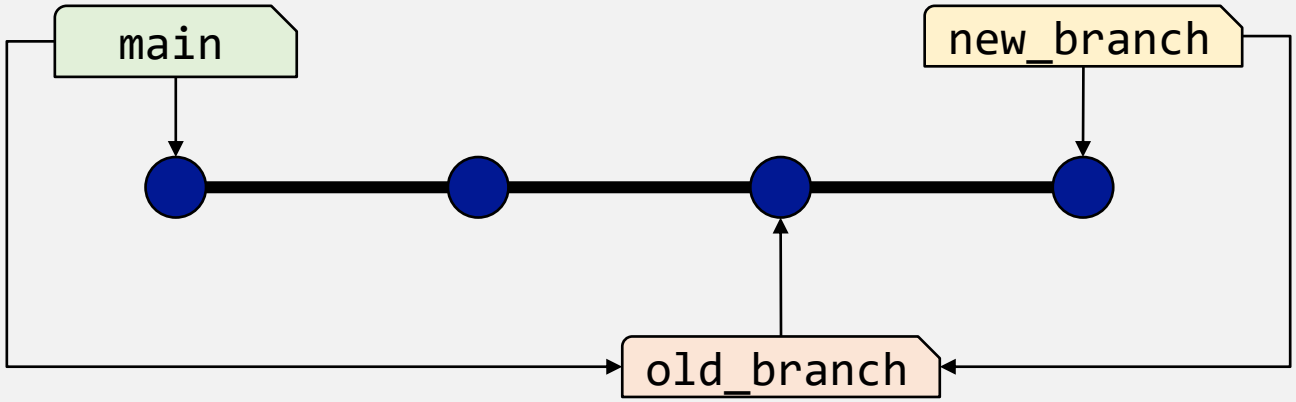
コミットは失われない
しかし、既存のブランチからたどれなくなる



各コミットは「親コミット」を覚えている
「親コミット」をたどることで自分のルーツを知ることができる
「子供コミット」は知らない
マージされていないブランチを削除すると、たどれないコミットが生じる

ブランチの削除

「マージされていないから削除できない」というエラーが出る条件は「カレントブランチ」からたどれるかどうか



mainからたどれないので
git branch –d old_branchができない

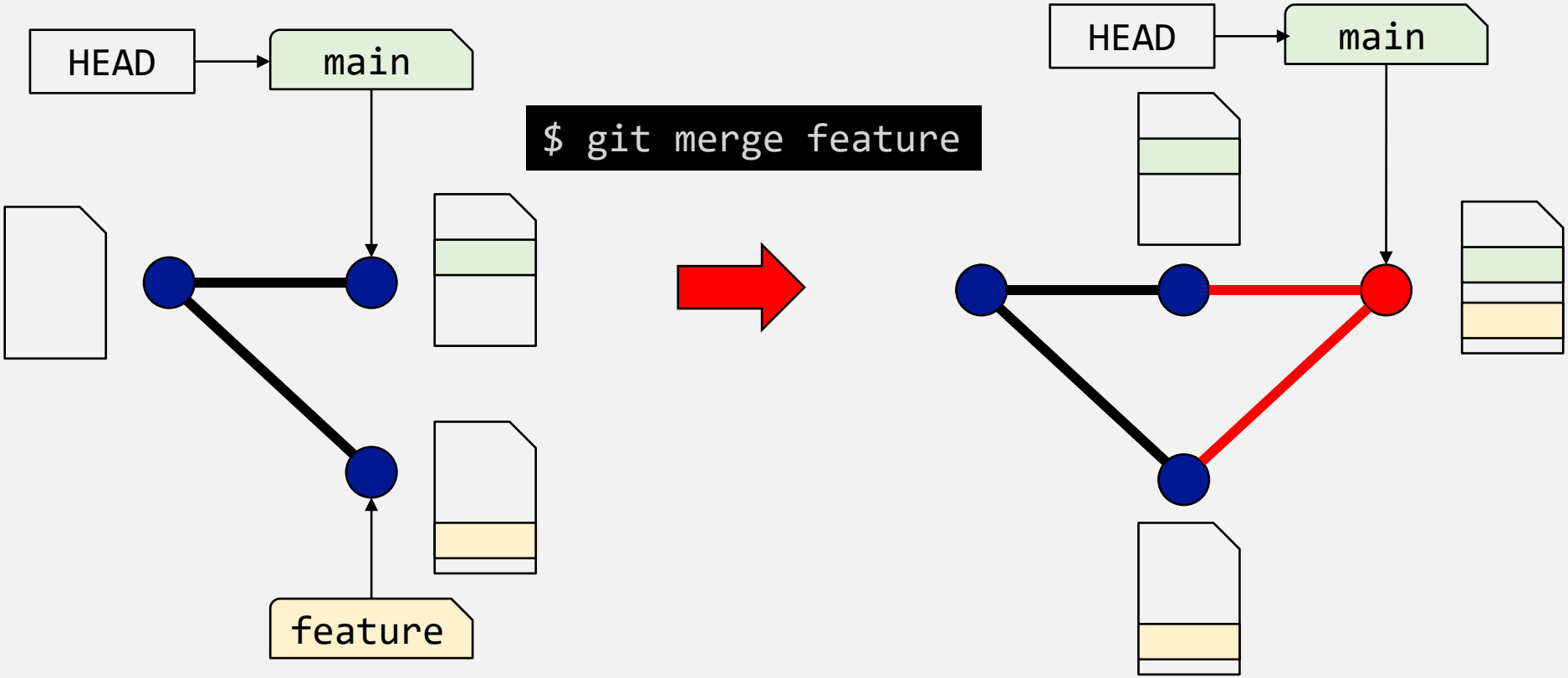
new_branchからたどれるので
git branch –d old_branchができる

```
$ git branch -d old_branch
```

カレントブランチがnew_branchの時は削除できる
カレントブランチがmainの場合は削除できない

衝突(コンフリクト)

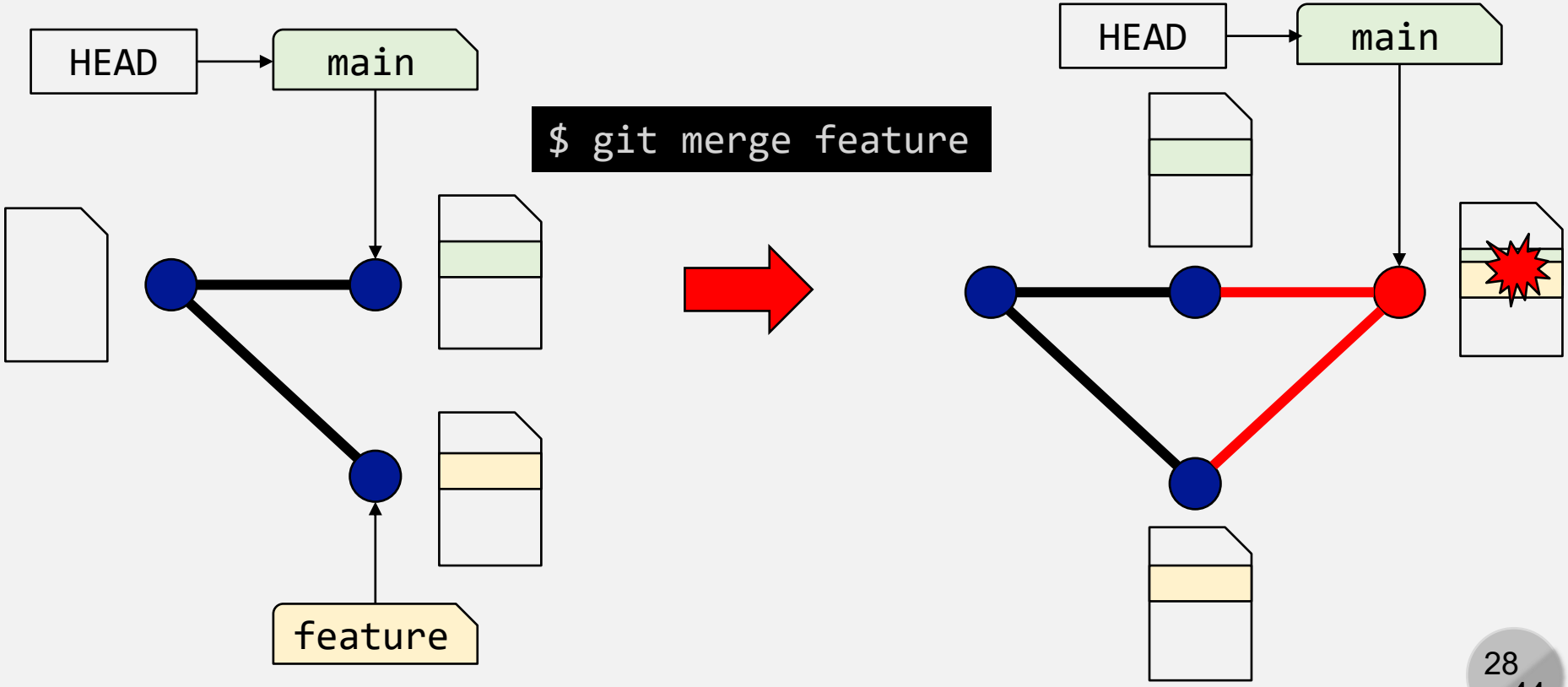
分岐した歴史をマージする際、Gitは可能な限り、両方の修正を取り込もうとする



異なるファイルが修正されていたら、両方の修正を取り込む
同じファイルでも、修正箇所が重なってなければ取り込む

衝突(コンフリクト)

同じファイルの同じ場所が同時に修正されていた場合、Gitはどのようにマージすれば良いか判断できない。これを衝突(conflict)と呼ぶ。



衝突(コンフリクト)

mainとfeatureで同時にtest.txtを修正した状態でマージしようとした

```
$ git merge feature
Auto-merging test.txt
CONFLICT (content): Merge conflict in test.txt
Automatic merge failed; fix conflicts and then commit the result.
```

- test.txtを自動マージしようとした
- 衝突(CONFLICT)が発生した
- 衝突を解決(fix)して、結果をコミットせよ

Gitは衝突を検出すると、ユーザに解決を求める

衝突(コンフリクト)

カレントブランチ(HEAD)でのtest.txt

```
Hello merge!

This line is modified on main.
```

featureブランチでのtest.txt

```
Hello merge!

This line is modified on feature.
```

マージに失敗し、衝突状態のtest.txt

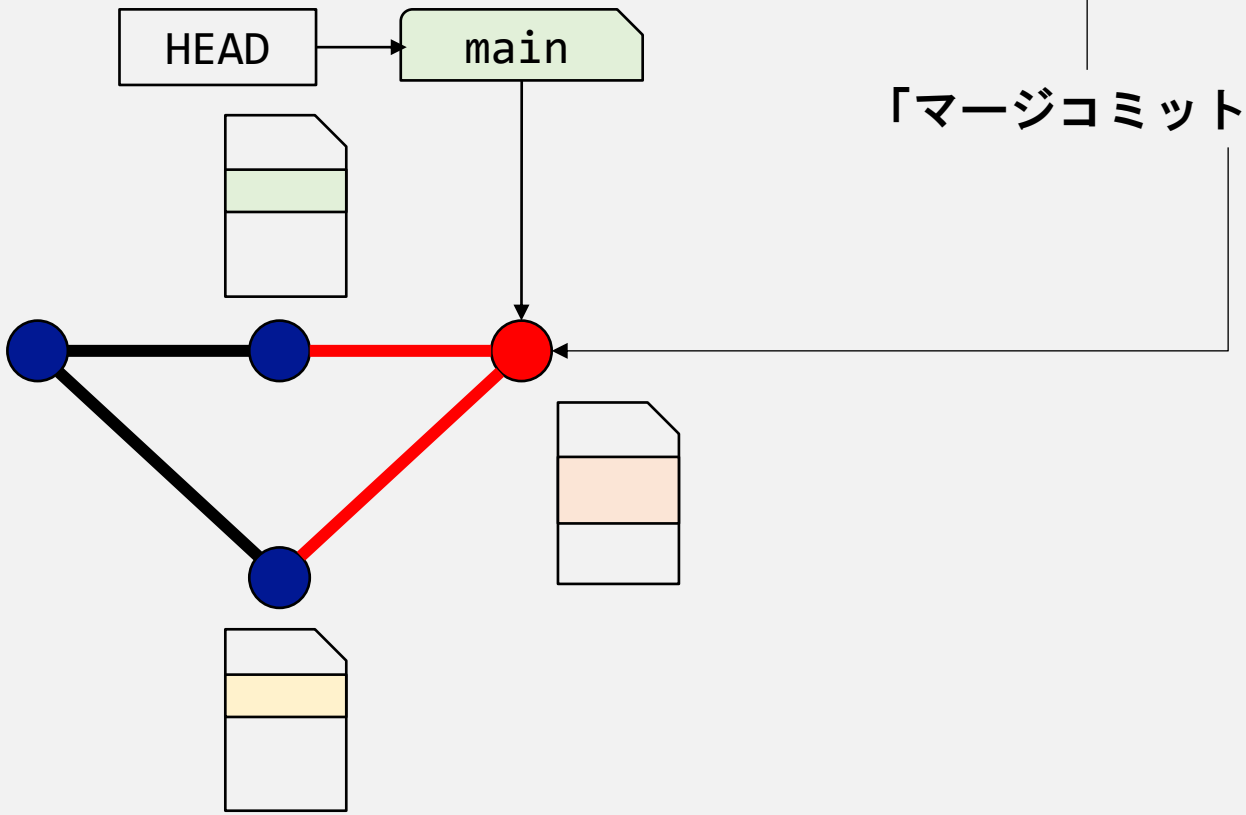
```
Hello Merge!

<<<<<<< HEAD
This line is modified on main.
=======
This line is modified on feature
>>>>>>> feature
```

ここがHEADではこうなってるけど

featureブランチではではこうなってるよ

衝突の解決

test.txtを適切に修正したあと、git add、git commitする



「マージコミット」を作っている

Gitが自動でマージコミットを作れなかったので、マージコミットのあるべき姿をインデックスに登録してコミット

マージと衝突のまとめ

マージとは何か

- 分かれた歴史を一つにまとめる作業
- 合流点を「マージコミット」と呼ぶ
- マージしたいブランチが直系の子孫である場合はブランチの移動でマージが完了する(fast forwardマージ)

衝突とその解決

- 自動でマージコミットを作れない状態を衝突と呼ぶ
- ユーザはマージコミットのあるべき姿をインデックスに登録、コミットする

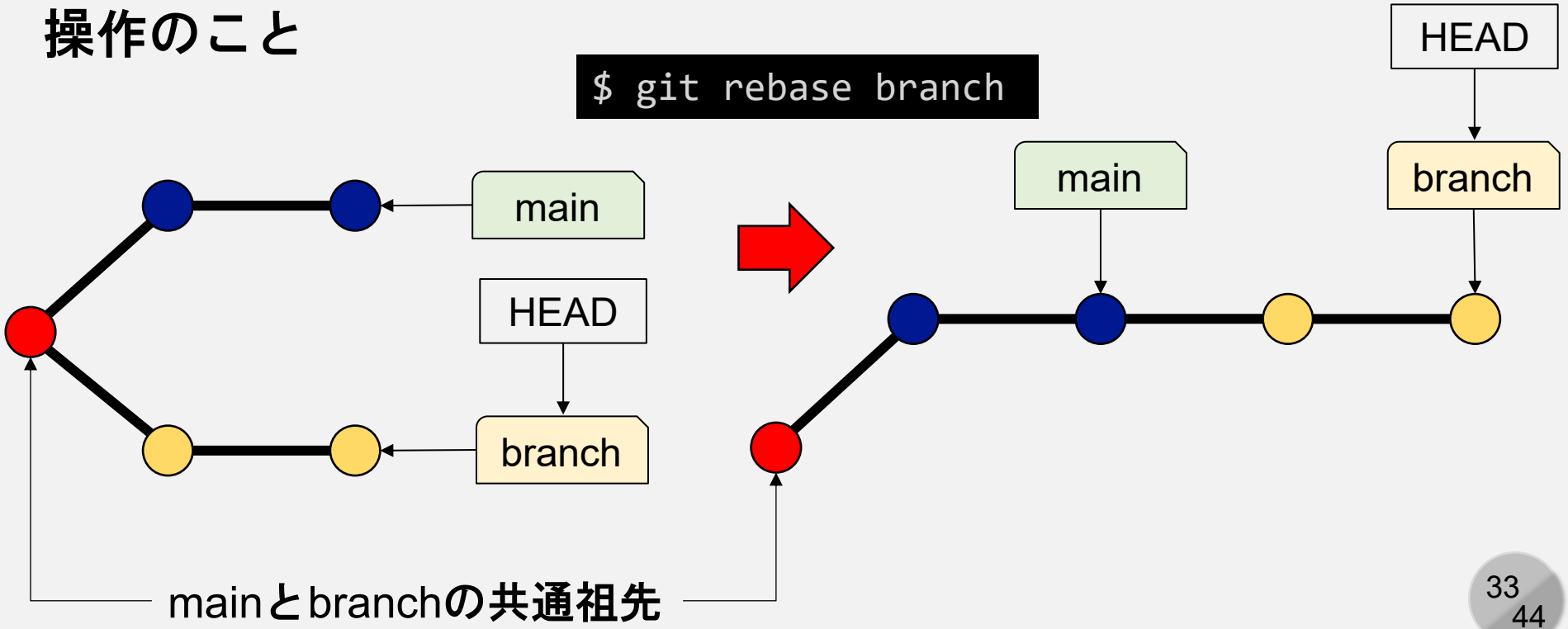
リベース

リベースとは

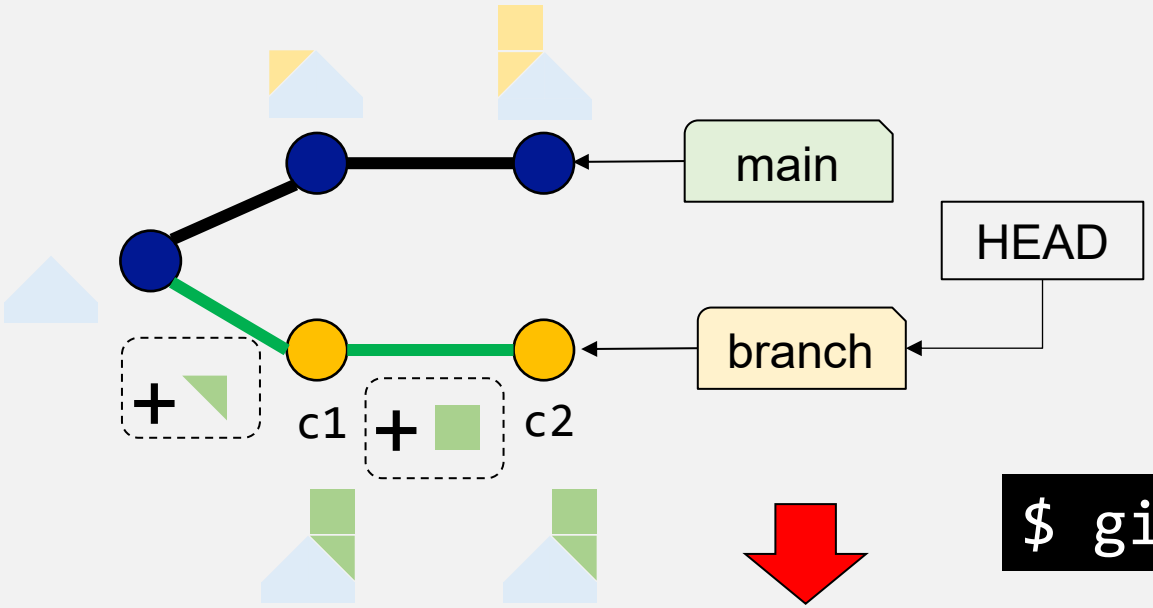
- カレントブランチとリベース先のブランチの共通祖先から
- カレントブランチまでの修正を
- リベース先のブランチにぶら下げる

操作のこと

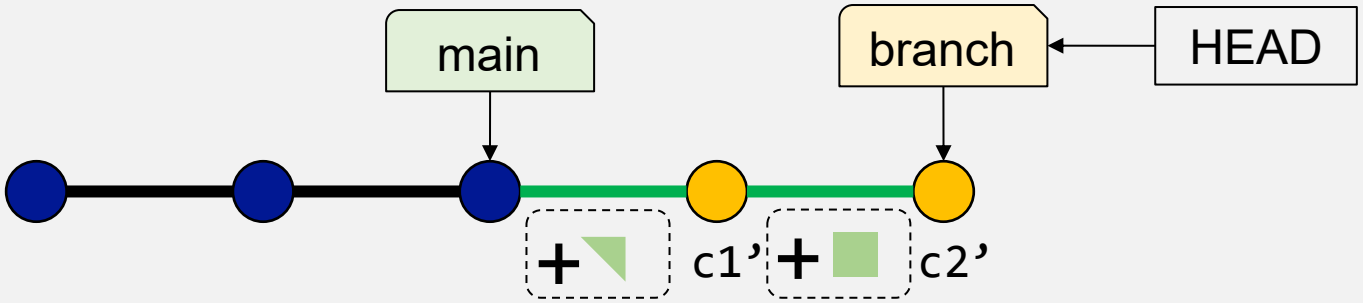
```
$ git rebase branch
```



リベース

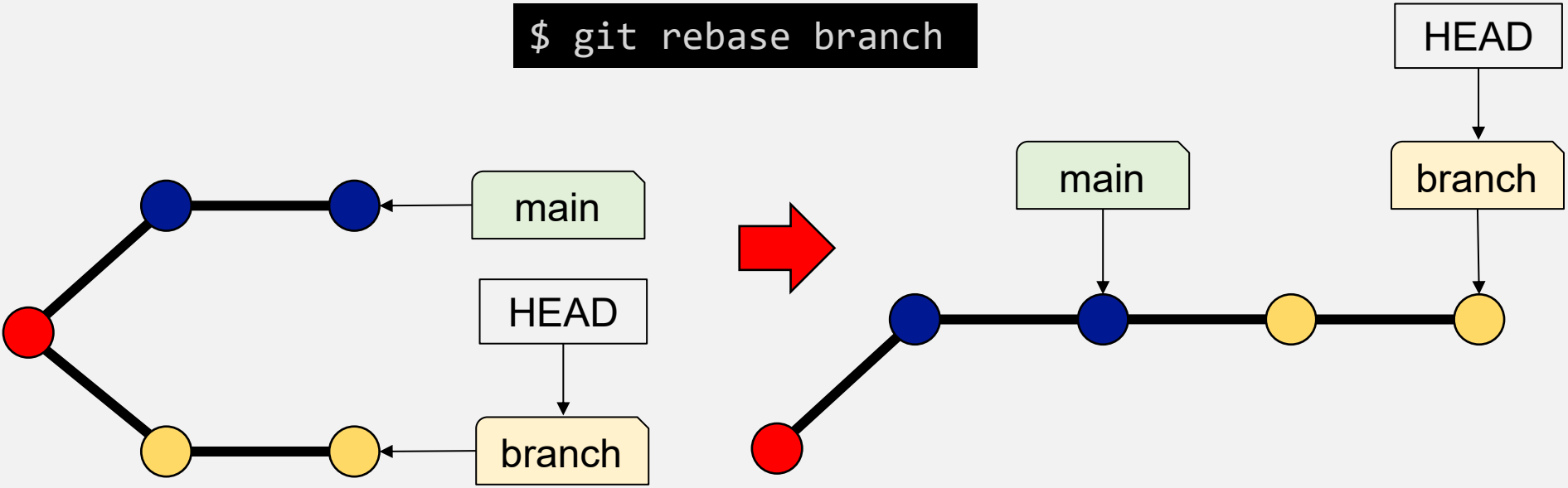



```
$ git rebase main
```



リベース

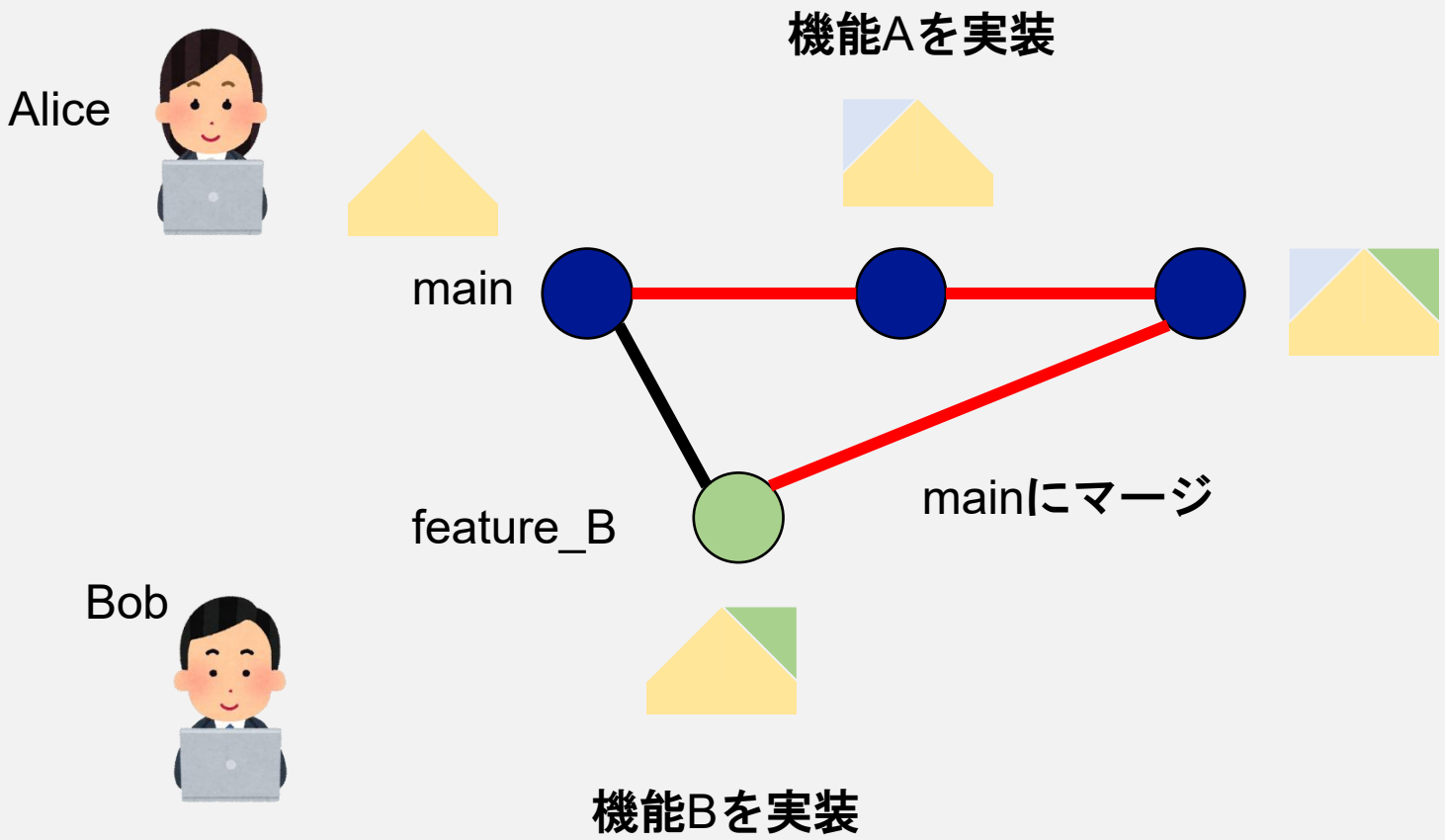
```
$ git rebase branch
```



branchブランチのベース(赤丸)を、mainブランチの指すコミットに張り替えたように見えるのでリベース
移動しているのは「線(パッチ)」であってコミットではない
コミットは新たに作られる

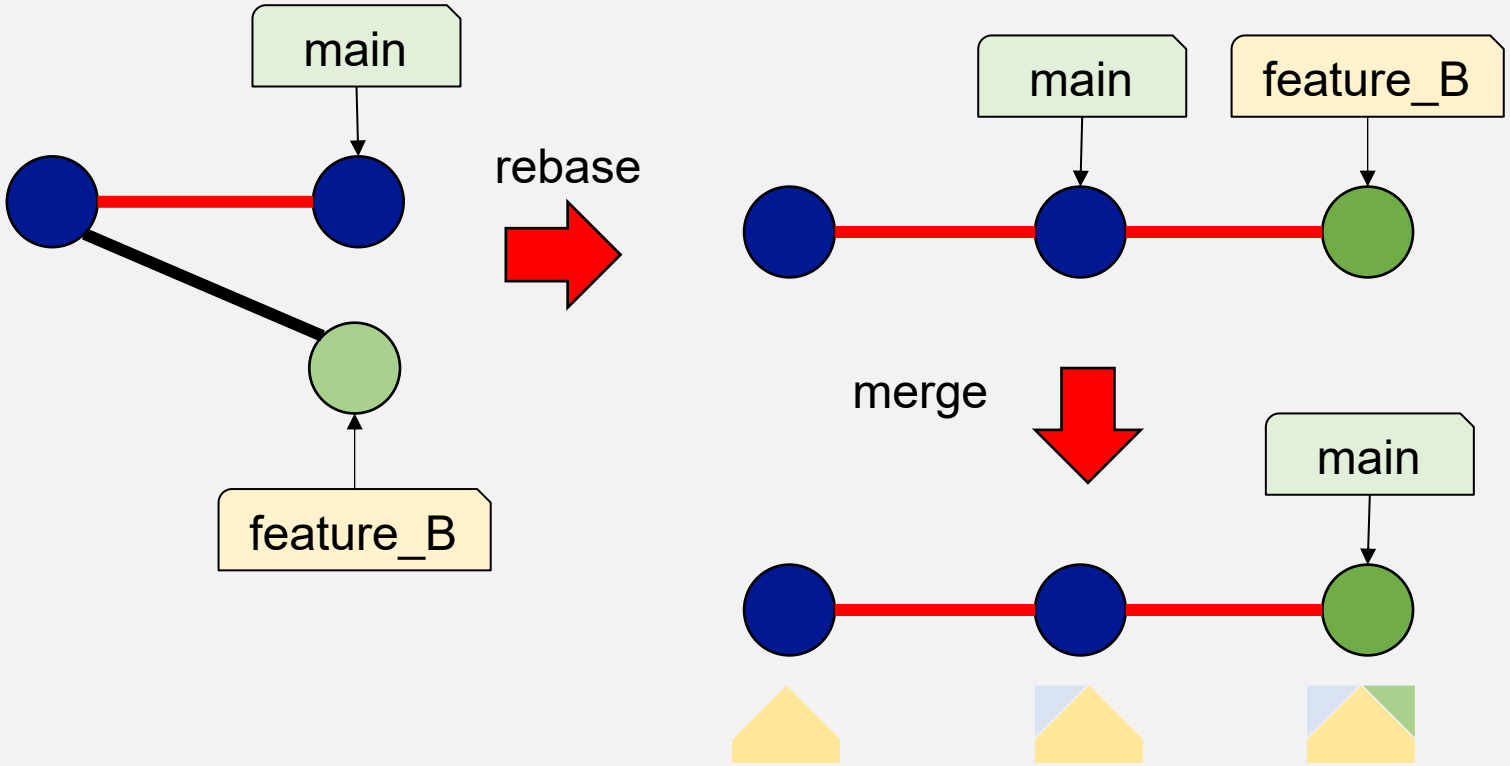
リベース

- mainブランチから、Aliceがfeature_Aを、Bobがfeature_Bを分岐
- Aliceが機能Aを実装してmainにマージ(fast-forward)
- Bobが機能Bを実装してmainにマージ(non-fast-forward)



リベース

- mainブランチから、Aliceがfeature_Aを、Bobがfeature_Bを分岐
- Aliceが機能Aを実装してmainにマージ(fast-forward)
- Bobが機能Bを実装してmainへリベース
- feature_Bがfast-forwardマージ可能になる



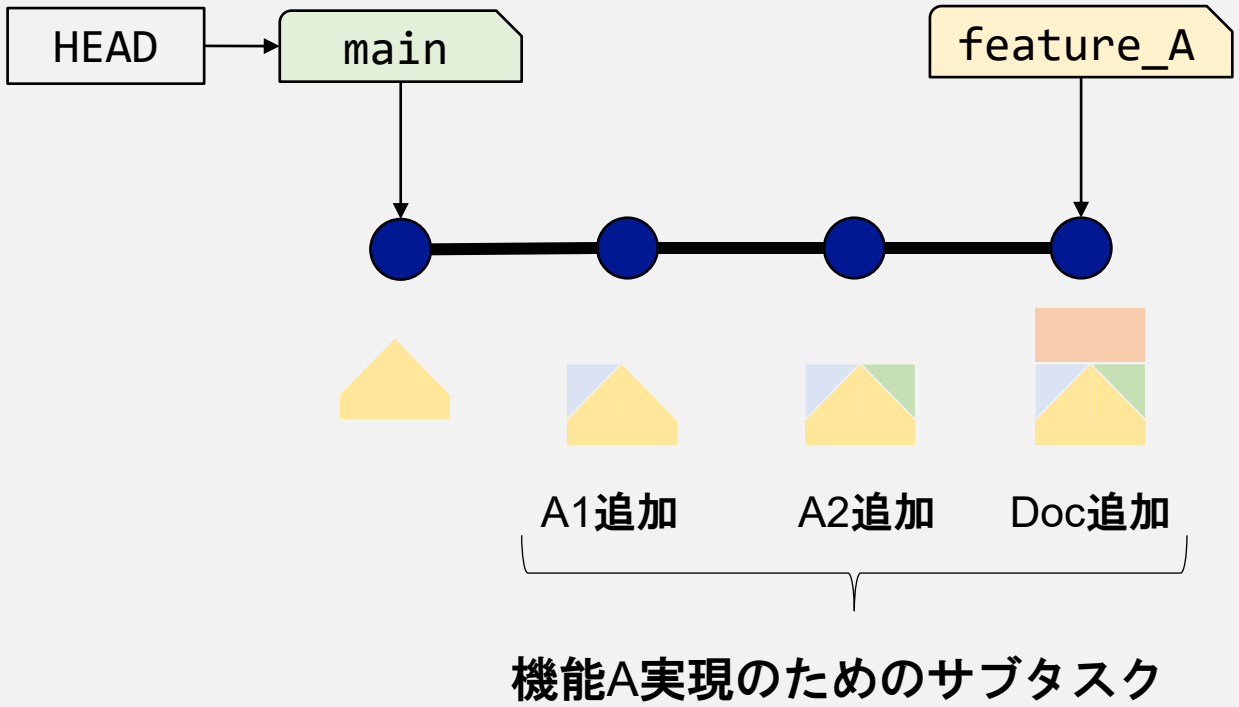
順番に開発したような歴史が作られた

機能A
を実装

機能B
を実装

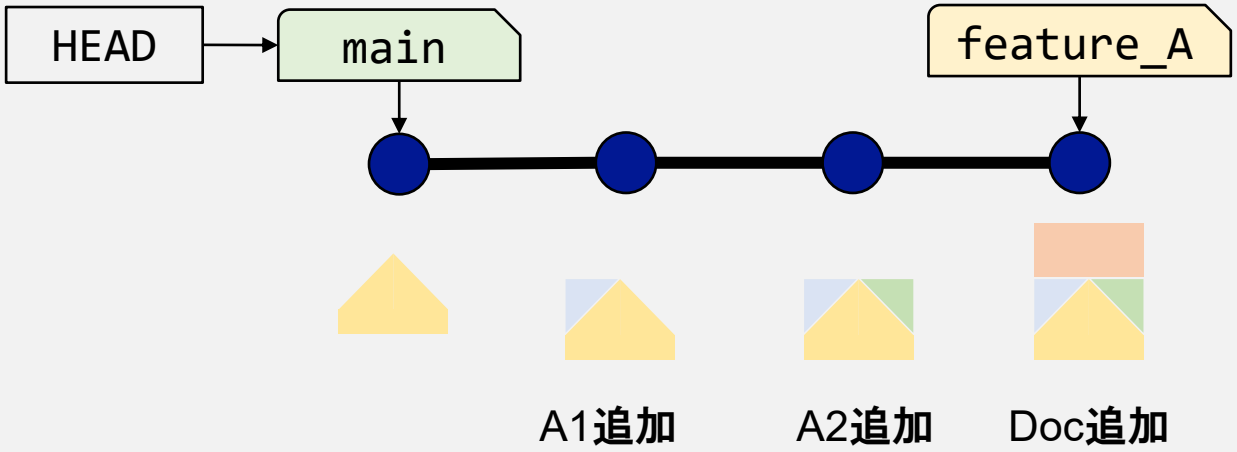
リベース

フィーチャーブランチをそのままmainにマージすると
「中途半端な状態のコミット」が残ることがある

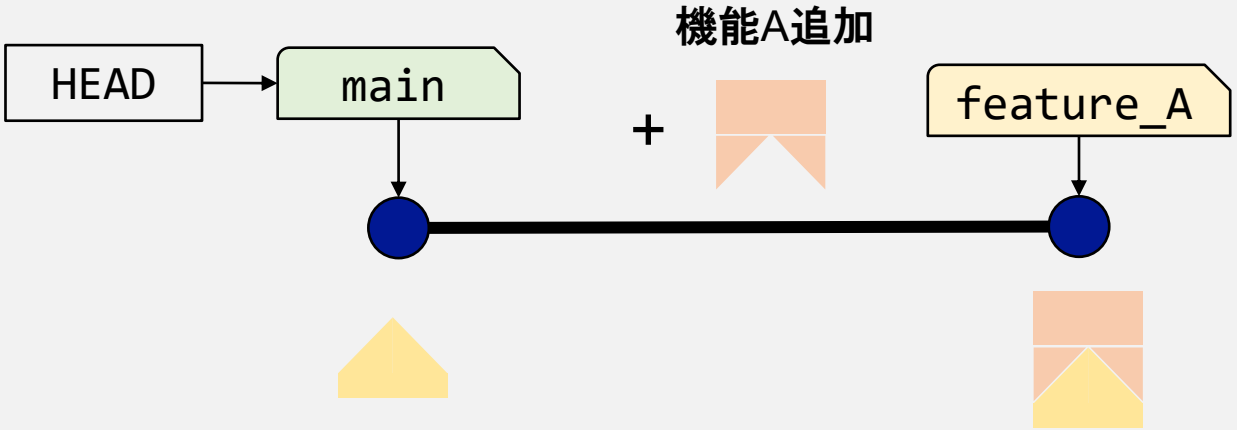


このままfast-forwardマージすると、「途中経過」も歴史に残る
mainブランチは「常にきれいな状態」であって欲しい

リベース



マージする前にコミットをまとめて歴史を整理しておく



歴史の整理にもリベースを使う

対話型リベース

```
$ git rebase -i ブランチ名
```

エディタが開き、以下のような画面がでる

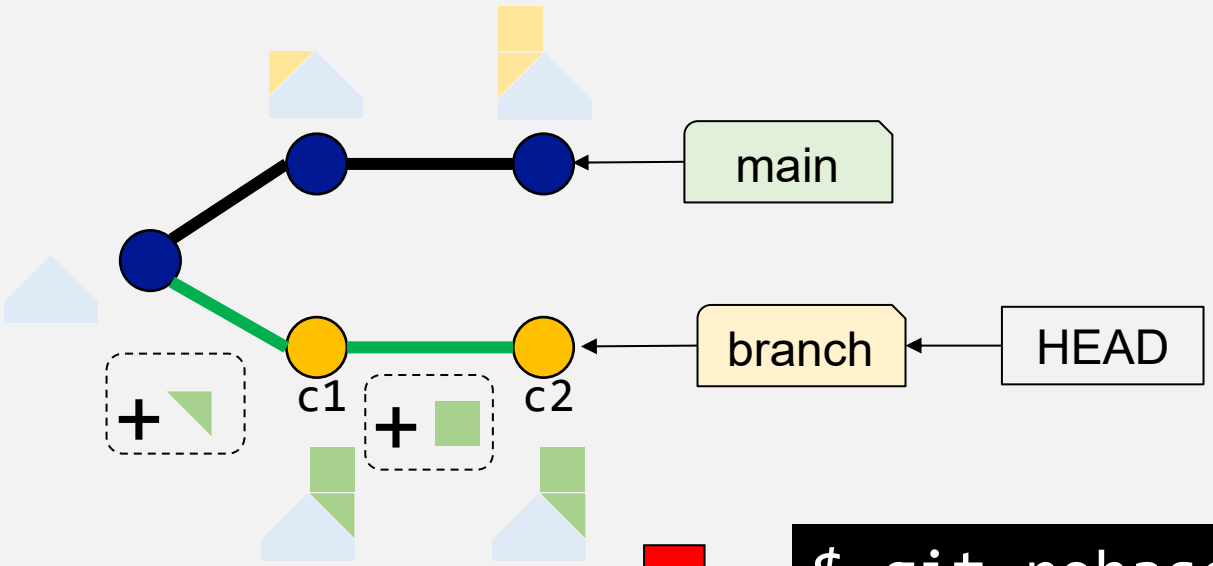
```
pick d6f185f c1
pick b2b0b0b c2

# Rebase e9c8c91..b2b0b0b onto e9c8c91 (2 commands)
```

リベースで移動するコミット候補が表示されており
どのコミットをどうするか選ぶ

pick　　: コミットをそのまま使う
squash: コミットは使うが、前のコミットとまとめる

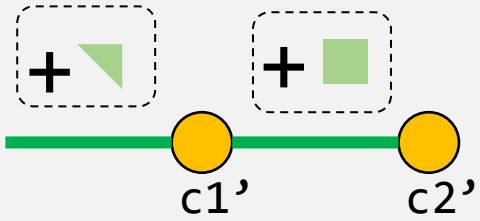
コミットを破棄したり、並べ替えたりもできる

コミットのsquash

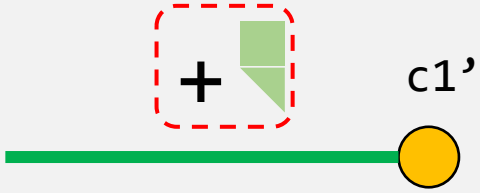



```
$ git rebase -i main
```

```
pick      c1      +
pick      c2      +
```



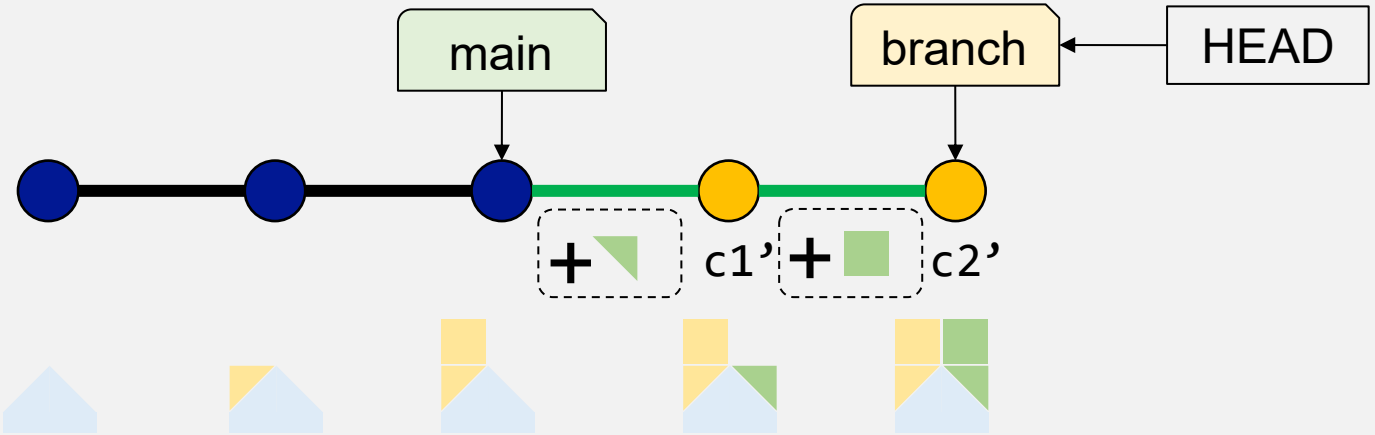

```
pick      c1      +
squash    c2      +
```



リベース中の衝突

リベースは「ベース」となるコミットに次々と
修正を適用すること操作

移動するコミットの数だけ衝突する可能性がある



「次に作られるべきコミット」をインデックスに
登録してコミットを繰り返す

リベースのまとめ

リベースとは、ブランチ(枝)の根本(ベース)を別の場所に移し替える操作
マージの前処理として行う
主に以下の歴史操作を行う
・分岐した歴史を一本にまとめる
・不要なコミットをまとめる(中間状態削除)

事故が起こりやすく、トラブルが起きた時の対処が難しいため、慣れるまでは使わなくて良い

まとめ

ワークフロー

Gitは原則としてmainブランチで作業せず、機能ごとにブランチを作って作業、後でマージする

マージと衝突の解決

マージは二つの歴史の修正をまとめたマージコミットを作る作業であり、衝突の解決とは「あるべきマージコミットの姿」をGitに教えること

リベース

リベースはマージの前処理として行う
「欲しい歴史」を作るために使う